

Codificação por Padrões

Serviços de Rede e Aplicações Multimédia

Mestrado em Engenharia de Telecomunicações e Informática

Ivo Miguel Menezes Silva

2º Semestre (25/26)

Adaptação dos slides originais do Prof. Bruno Dias (Dept. Informática – UMinho)

Codificação por Padrões (ou por Dicionário)

Família de algoritmos utilizados inicialmente para compressão de fontes de informação do tipo textual.

Variantes modernas comprimem **qualquer tipo de informação** sem perdas (*lossless*).

- ✓ **Não necessita** informação estatística prévia dos símbolos.
- ✓ A tabela de padrões é o **dicionário**.
- ✓ **Estratégia genérica de codificação**: encontrar padrões no texto e substituir os padrões por códigos.
- ✓ As variantes modernas são usadas na codificação sem perdas de informação
- ✓ Também chamada compressão por dicionário
- ✓ Atualmente usados em diversos formatos: ZIP, GIF, TIFF, e muitos protocolos de rede modernos

Codificação por Padrões

Vantagens Principais ✨

- ✓ **Compressão:** Níveis equivalentes aos obtidos com Shannon-Fano adaptativa com blocos seletivos ($K > 1$)
- ✓ **Simplicidade:** Algoritmos fáceis de implementar, com funções matemáticas simples e de rápida execução
- ✓ **Poucos recursos computacionais:** Algumas variantes requerem poucos recursos computacionais, por isso são práticas para implementação em equipamentos com poucos recursos computacionais.
- ✓ **Simetria dos algoritmos:** mecanismo de descodificação é quase igual ao de codificação (algoritmos simétricos).

Tipos de Dicionários

Pelo comportamento

- Adaptativos
- Semi-adaptativos
- Estáticos

Pelo tipo

- Implícitos
- Explícitos

Dicionários adaptativos

Os padrões mais **longos vão sendo incluídos no dicionário**, substituindo os mais curtos. O dicionário **evolui dinamicamente à medida que a informação é codificada**.

Características

- Padrões mais longos incluídos no dicionário à custa de padrões mais curtos
- Estratégia MAIS COMUM

Exemplos

- LZ77
- LZ78
- LZSS
- LZW

Tipos de Dicionários

Semi-adaptativos

- Padrões básicos incluídos à partida no dicionário
- Quando se incluem inicialmente padrões mais compridos é porque se sabe o tipo de informação a codificar (p. ex., texto em inglês)

Estáticos

- Todos os padrões conhecidos à partida
- Nenhuma adaptação durante a codificação
- Mais simples, mas menos flexível

Tipos de Dicionários: Gestão do Dicionário 🧹

Sem limpeza

- O dicionário pode conter um número máximo de padrões.
- Quando o número máximo ou o número total de símbolos da fonte é atingido, a codificação reinicia-se com um dicionário vazio/inicial.
- Estratégia mais comum.

Com limpeza

- Quando o número máximo de entradas é atingido, os padrões já incluídos no dicionário e, menos utilizados, vão sendo substituídos por padrões novos.
- Estratégia menos comum e mais complexa de implementar.

Tipos de Dicionários: Redirecionamento

Sem redirecionamento

- Existe apenas um dicionário que pode conter um número máximo de padrões.
- Estratégia mais comum e mais simples.

Com redirecionamento

- É quando existe mais do que um nível de dicionário.
- Em cada nível mais alto de redirecionamento é utilizado um dicionário mais pequeno apenas com os padrões mais utilizados do dicionário maior de nível anterior.
- Técnica significativamente mais complexa, mas permite diminuir o tamanho médio dos códigos gerados.

Procura de Padrões no Dicionário

Procura de Padrões

Processo computacionalmente mais exigente desta família de algoritmos.
Consiste em 2 fases:

Fase i) Verificação

Verificação se um conjunto de símbolos ainda não codificados já pertence ao dicionário.

Fase ii) Inclusão

Inclusão no dicionário do **NOVO** padrão encontrado.

Estratégias de Procura

Longa, conservadora e ótima.

Procura de Padrões no Dicionário

Procura longa

Tenta encontrar o padrão mais comprido ainda não existente no dicionário e que seja possível definir à custa dum padrão muito parecido do dicionário (em geral por diferença de um símbolo).

Procura conservadora

Parecida com a procura longa, mas a procura é limitada a uma zona do dicionário. É mais rápida de implementar mas tem menor rendimento.

Procura ótima

Procura o padrão maior possível definido como uma combinação de vários padrões anteriormente definidos. É o mais complexo e lento de implementar mas pode atingir níveis de rendimento ainda melhores.

Procura de Padrões no Dicionário: Metodologias Alternativas

↑ Algoritmos clássicos de *sorting*

Estratégias comuns e fáceis de implementar (árvores B, árvores binárias, árvores Patricia e *Compact Directed Acyclic Word Graph* são muito utilizadas neste contexto).

Tabelas de *hash*

Estruturas de dados que armazenam pares de **chave-valor**, permitindo buscas, inserções e remoções extremamente rápidas, quase em tempo constante, ao usar uma função *hash* para calcular diretamente o índice de armazenamento de cada item, em vez de percorrer uma lista. Também são muito utilizadas e fáceis de implementar.

Arrays de índices

Estruturas de dados que armazenam coleções de elementos (índices) de forma ordenada numa única variável, onde cada elemento é identificado por um índice numérico, permitindo acesso rápido e manipulação eficiente desses dados. São menos comuns e, por isso, mais difíceis de implementar, mas são muito eficientes, sobretudo quando é preciso implementar procuras longas.

Codificação por Padrões:

Técnica de compressão Run-Length Encoding (RLE)

Run-Length Encoding (RLE)

- ✓ Técnica simples de compressão *lossless* onde sequências longas de valores repetidos são armazenadas como um único valor e sua contagem no lugar de sua sequência original.
- ✓ Tem bom rendimento para repetições simples do mesmo símbolo quando aparece muitas vezes em fontes de informação digitais não processadas (bases de dados, dados multimédia, etc.).
- ✓ Não é eficiente para compressão de textos.
- ✓ Pode ser aplicada independentemente do tipo de símbolo (*bit, byte, word, etc.*) definindo-se um padrão de codificação duma sequência de até N repetições do mesmo símbolo.
- ✓ O padrão de codificação usa um marcador especial.

Codificação por Padrões: RLE Binário

RLE Binário 0 1

- Utilizado quase exclusivamente em protocolos de baixo nível.
- Utiliza-se um código para indicar o número de bits repetidos consecutivamente.
- As duas variantes mais comuns são:
 - Uma com marcador explícito apenas quando acontecem, no mínimo, m repetições de 1s ou de 0s;
 - Outra que não utiliza marcadores explícitos mas que indica sempre explicitamente o número de 1s e 0s alternadamente; é mais simples/rápida de implementar mas só tem bom rendimento em comunicações com fontes que geram informação aos “molhos”.



EXERCÍCIO: RLE Binário (marcador explícito)

Marcador: **1111**, $m = 4$ (comprimento marcador), sequências de 1s: número de repetições = $R + 4$, para sequências de 0s: número de repetições = $R + 9$

Este marcador identifica quando começa uma codificação RLE

Código: **1111** R

R é o número de repetições em binário;

A codificação de R pode ser direta com um n.º fixo de bits ou uma codificação com tamanho adaptativo, como por exemplo a codificação Elias Gamma.

Sequência: 1000000000111111111111111111011110



EXERCÍCIO: RLE Binário (marcador explícito)

Marcador: **1111**, $m = 4$ (comprimento marcador) 1s ou 9 0s

Este marcador identifica quando começa uma codificação RLE

Código: **1111** R

R é o número de repetições em binário;

A codificação de R pode ser direta com um n.º fixo de bits ou uma codificação com tamanho adaptativo, como por exemplo a codificação Elias Gamma.

Sequência: 100000000011111111111111111011110



EXERCÍCIO: RLE Binário (marcador implícito)

$m = 1$ (alternadamente para 1s e 0s)

Este marcador identifica quando começa uma codificação RLE

Código: *RRR...*

R é o número de repetições em binário;

A codificação de R com codificação *Elias gamma* (código indica quantas vezes o bit se repete).

Sequência: 100000000111111111111111111011110

Codificação Elias gamma

Para um inteiro positivo n :

1. Escreve-se n em binário (sem zeros à esquerda).
2. Conta-se o número de bits k .
3. Escrevem-se $k-1$ zeros.
4. A seguir, escreve-se o binário de n .

Tabela apresenta exemplos de vários valores.

Valor (n)	Binário	Bits (k)	Zeros (k-1)	Código Final
1	1	1	(nenhum)	1
5	101	3	00	00101
10	1010	4	000	0001010
19	10011	5	0000	000010011



EXERCÍCIO: RLE Binário (marcador implícito)

$m = 1$ (alternadamente para 1s e 0s)

Este marcador identifica quando começa uma codificação RLE

Código: $RRR...$

R é o número de repetições em binário;

A codificação de R com codificação *Elias gamma* (código indica quantas vezes o bit se repete).

Sequência: 100000000011111111111111111011110

Codificação por Padrões: RLE por Bytes

RLE por Bytes 0 1 1 0 1 0 0 1

- É o tipo mais utilizado em aplicações ou protocolos aplicacionais.
- Usa-se sempre um marcador M explícito (às vezes designado por *escape symbol*), e que normalmente é o *byte* com o valor zero.
- É definido um valor m de repetições mínimas, exceto para o caso do próprio símbolo marcador (nesse caso é sempre gerado um código RLE).
- Os códigos são $MX_iR_0R_1\dots R_n$ para cada $m + \sum_{j=0}^n R_j$ repetições do símbolo X_i (define-se um valor fixo de R_j para indicar que existe mais um R_{j+1}).

EXERCÍCIO: RLE por Bytes

Marcador: #, $m=4$ (n.º de repetições mínimas)

Código: $\#X_i R_0 R_1 \dots R_n$ (cada R_j codificado num byte)

O valor fixo de R_j para indicar que existe $R_{(j+1)}$ é {255}

Sequência: aabbbbcbdaaaaaaaaaaabbccddddddcb

Codificação por Padrões: Codificação LZ

? O que é?

Família de protocolos com dicionários adaptativos de padrões LR, introduzidos na década de 1970 por Abraham Lempel e Jacob Ziv.

Os algoritmos dividem-se em dois grandes grupos de estratégias: **LZ77** e **LZ78**.

LZ77

📖 **Dicionário implícito** na parte já processada da sequência original (janela de procura).

📅 Todos os padrões LR (*Left-to-Right*) dentro da janela são possíveis.

LZ78

📖 **Dicionário explícito** que se constrói dinamicamente à medida que os padrões LR vão sendo descobertos e que são definidos à custa de padrões mais simples.

🔧 Costumam ser mais simples e rápidos de implementar mas têm menor rendimento de compressão.

Codificação LZ77

A estratégia básica é gerar códigos todos iguais que são valores triplos (*offset*, *length*, *symbol*). Cada valor individual do triplo é codificado no menor número de bits possível, dependendo do tamanho da janela de procura.

Variantes mais comuns

LZH, LZR, LZSS, LZB, Deflate, LZMA, GZIP

Aplicações:

- LZH: zip e unzip
- Deflate, LZMA, GZIP: são dos algoritmos mais recentes e mais eficientes que existem para compressão sem perda de informação (combinam variações do LZ77 original com Huffman) e são utilizados em aplicações como WinZip, 7-Zip e GZip

Codificação LZ77 (Janela de Procura)

- O *offset* é a distância negativa entre o ponto C em que a codificação se encontra e o início do padrão P mais longo que é igual à sequência à frente de C ;
- O *length* é o comprimento do padrão P ;
- O *symbol* é o primeiro símbolo à frente do padrão P após C .

- Exemplo (janela de procura de 19 símbolos, a azul):

...2526412344455668901234571234509836568576... $\rightarrow (6,5,0)$

Neste caso, o ponto C divide a informação entre o símbolo 7 e o símbolo 1 à sua frente/direita e o padrão maior que coincide com a sequência à frente de C é 12345 e se inicia a seis posições negativas (à esquerda) do ponto C .

- O código gerado é $(6,5,0)$ e a janela e C avançam $length+1$ símbolos para a frente/direita. ($offset=6, length=5, symbol=0$)

Codificação LZ78

Com a possibilidade de se conhecer o tipo de dados de antemão, esta família de algoritmos pode ser bastante otimizada com a utilização de dicionários iniciais muito completos.

Também podem utilizar-se técnicas e estruturas de dados de procura de padrões muito eficientes.

Variantes mais comuns

- LZW, LZC, LZT, LZMW, LZJ e LZFG

Aplicações

- LZW, LZC, LZT, LZMW, LZJ: são a base para aplicações como o compress, formatos como o GIF ou técnicas de compressão usadas em antigas normas CCITT para compressão em modems V.42.
- LZFG: algoritmo mais recente e mais eficiente desta família, só batido marginalmente pelo GZIP (variante de LZ77)

Codificação LZ78

Dicionário explícito (adaptativo) que se vai construindo com os padrões encontrados.

A estratégia consiste em gerar códigos que são valores tuplos do tipo $(index, symbol)$, em que $index$ é o índice do padrão na tabela de padrões e $symbol$ é o primeiro símbolo da sequência por codificar e que não pertence ao padrão.

Não é definida qualquer janela mas é definido um tamanho máximo T para a tabela de padrões. Quando esse máximo é atingido a tabela é reiniciada com zero entradas.

Dependendo do tamanho máximo permitido para a tabela de padrões a codificação do valor de $index$ vai precisar de mais ou menos bits ao passo que $symbol$ precisa sempre do mesmo número de bits. Geralmente $T=4096$, pelo que cada tuplo ocupa $12+8=24$ bits, sem otimizações adicionais.



EXEMPLO: Codificação LZ78

Sequência para codificar:

abcabcdaabbccaaabbb

Codificação LZW

Algoritmo de compressão de dados sem perdas, baseado em dicionário, amplamente utilizado em formatos como GIF, TIFF, PDF e ZIP/RAR.

Funciona substituindo padrões de dados repetidos por códigos de tamanho fixo (geralmente 12 bits), construindo dinamicamente um dicionário durante a compressão sem necessidade de análise prévia.

Características:

- Ao contrário do LZ78, o LZW gera códigos simples do tipo (*index*), que são apenas os índices dos padrões encontrados. Com tabelas de um máximo de 4096 entradas cada índice pode ser codificado em grupos de **12 bits**, no máximo, sem otimizações adicionais.
- Porque apenas são gerados códigos de índices da tabela com padrões já conhecidos, a tabela tem de ser iniciada com todos os padrões simples com apenas um símbolo (se os símbolos forem *bytes* então a tabela inicia-se com 256 entradas).

Codificação LZW

Processo de codificação

Lê os dados, procura a maior sequência correspondente no dicionário, emite o código dessa sequência e adiciona a nova sequência (anterior + próximo caractere) ao dicionário.

Decodificação

O decodificador reconstrói o mesmo dicionário dinamicamente lendo os códigos, sem precisar da tabela criada pelo codificador.



EXEMPLO: Codificação LZW

Sequência para codificar:

abcabcdaabbccaaabbb

Variante experimental UMinho: LZW-HX

Considere-se a sequência de símbolos $S = S_1 S_2 \dots S_N$ na entrada de dados, em que cada símbolo S_i pode assumir um de K valores possíveis de um alfabeto $A = \{X_1, X_2, \dots, X_K\}$. Por conveniência, $S[i] = S_i$ e $A[i] = X_i$. Defina-se um dicionário com um máximo de T padrões tal que $D = \{P_0, P_1, \dots, P_{T-1}\}$, em que $D[i] = P_i$ é o padrão identificado pelo índice i .

Valores típicos: $K = 2^8 = 256$, $T = 2^{16} = 65536$.

O dicionário inicial contém:

- Todos os **padrões de um símbolo**: os K símbolos do alfabeto (normalmente os 256 bytes possíveis), ocupando as posições 0 a $K - 1$ do dicionário.
- Um conjunto adicional de **padrões de dois símbolos pré-definidos**, que dependem do tipo de ficheiro a processar. Estes padrões ocupam as posições K a $K + M - 1$, em que M é o número de padrões de dois símbolos. A inicialização do dicionário com os padrões de dois símbolos é uma opção configurável pelo utilizador, se estiver desativada, o dicionário é inicializado com os padrões de um símbolo.

Em cada passo, o LZW-HX processa a sequência de entrada em torno de um par de padrões consecutivos $P_a | P_b$, definidos como os **maiores padrões conhecidos** que encaixam na posição corrente.

Variante experimental UMinho: LZW-HX

O algoritmo de codificação procede da seguinte forma:

1. A partir da posição atual em S , encontrar o maior padrão conhecido P_a que coincida com a sequência de símbolos nessa posição. Se não houver correspondência com nenhum padrão de comprimento superior a 1, P_a é o símbolo individual nessa posição.
2. Imediatamente após o fim de P_a , encontrar o maior padrão conhecido P_b que coincida com a sequência de símbolos seguinte. Se não houver correspondência, P_b é o símbolo individual nessa posição.
3. Enviar para o **buffer de saída (ou ficheiro)** o código de P_a , de acordo com as regras de codificação definidas.
4. Atualizar o dicionário aplicando as **regras do LZW-HX**.
5. Avançar o processamento para o início de P_b (ou seja, a posição após o fim de P_a) e repetir os passos 1 a 4 enquanto existirem símbolos por processar em S .
6. Quando não houver mais símbolos em S após P_b , enviar para o buffer/ficheiro o código de P_b (ou do último padrão encontrado) e terminar a codificação.

Variante experimental UMinho: LZW-HX

NOTAS IMPORTANTES:

NOTA 1: Se tivermos já no dicionário os seguintes padrões: "A", "B", "C", "D", "AA" e "BB" e se num determinado passo encontrarmos $P_a = \text{"AA"}$ e $P_b = \text{"BB"}$, então o novo padrão a considerar é "AABB". Neste caso a inserção de "AABB" é para substituir o padrão "A" ou "AA", uma vez que ambos são prefixos de "AABB". Nestas situações, substitui-se o padrão do dicionário que pelo padrão cujo prefixo que tiver mais símbolos em comum com o padrão a considerar, "AABB". Neste caso seria o "AA", uma vez que tem dois símbolos em comum.

NOTA 2: Nas situações em que P_a ou P_b são sub-padrões de um padrão já existente e já existam vários padrões possíveis, cujo prefixo pode ser usado para representar P_a ou P_b , é necessário escolher qual dos padrões é usado.

Por exemplo, considerando um dicionário com os padrões: "A", "B", "AA", "BB", "BA", "ABBA", "ABAB" e se encontrarmos $P_a = \text{"AB"}$, este padrão pode ser representado pelos prefixos representados em "ABBA" e "ABAB". Nestas situações, P_a pode ser representado pelo padrão do dicionário que já tenha mais utilizações até ao momento.

Variante experimental UMinho: LZW-HX

NOTAS IMPORTANTES:

NOTA 3: Os padrões no dicionário que sejam maiores que um símbolo, permitem a identificação de sub-padrões que são seus prefixos, sem haver a necessidade de inserir esses sub-padrões como padrões individuais no dicionário. Quando é preciso enviar para a saída um código que identifique um padrão utiliza-se apenas o valor do seu índice se esse padrão for um padrão inserido no dicionário e tem apenas um símbolo; utiliza-se o valor do índice e o tamanho do padrão se o padrão tem mais de um símbolo ou, tendo apenas um símbolo, faz parte doutro padrão maior.

Exemplo: se existirem no dicionário $P_{23} = \text{"AAAC"}$ e $P_{34} = \text{"BBC"}$. Se houver a sequência de entrada $P_a = \text{"AAA"}$ e $P_b = \text{"BB"}$ (e ainda não existirem os padrões "AAA" ou "BB" no dicionário), ainda que sejam ambos prefixos dos padrões que realmente existem no dicionário. Quando for para enviar o seu código para a saída, o código de P_a será igual ao tuplo formado pelo seu índice e pelo seu tamanho, i.e., (23,3) e o código de P_b será (34,2).



EXEMPLO: Codificação LZW-HX

Considere-se uma sequência simplificada com alfabeto $K = 2$ (símbolos "A" e "B"):

"AAABABBABABAABBABBAB"

Dicionário inicial (inclui padrões de 1 e 2 símbolos):

$D[0] = \text{"A"}, D[1] = \text{"B"}, D[2] = \text{"AA"}, D[3] = \text{"BB"}, D[4] = \text{"AB"}, D[5] = \text{"BA"}$

Sequência a codificar ($K=2$): "AAABABBABABAABBABBAB"

Passo 1a: $P_a = \text{"AA"} = D[2]$ e $P_b = \text{"AB"} = D[4]$

Passo 1b: Enviar para a saída código de P_a : (2)

Passo 1c: Padrão candidato $C_1 = \text{"AAAB"} \rightarrow$ Pode ocupar lugar de "AA", então $D[2] = \text{"AAAB"}$

Passo 1d: $P_a = P_b$

Passo 2a: $P_a = \text{"AB"} = D[4]$ e $P_b = \text{"AB"} = D[4]$

Passo 2b: Enviar para a saída código de P_a : (4)

Passo 2c: Padrão candidato $C_1 = \text{"ABAB"} \rightarrow$ Pode ocupar lugar de "AB", então $D[4] = \text{"ABAB"}$

Passo 2d: $P_a = P_b$



EXEMPLO: Codificação LZW-HX

Sequência a codificar (K=2): "AAABABBBABABAABBABBBAB"

Passo 3a: Pa="AB"=D[4], 2 e Pb="BA"=D[5]

Passo 3b: Enviar para a saída código de Pa: (4,2)

Passo 3c: Padrão candidato C1="ABBA" -> Novo padrão, então D[6]="ABBA"

Passo 3d: Pa=Pb

Passo 4a: Pa="BA"=D[5] e Pb="BA"=D[5]

Passo 4b: Enviar para a saída código de Pa: (5)

Passo 4c: Padrão candidato C1="BABA" -> Pode ocupar lugar de "BA", então D[5]="BABA"

Passo 4d: Pa=Pb

Passo 5a: Pa="BA"=D[5], 2 e Pb="BA"=D[5], 2

Passo 5b: Enviar para a saída código de Pa: (5,2)

Passo 5c: Padrão candidato C1="BABA" -> Já está definido em D[5]

Passo 5d: Pa=Pb

Passo 6a: Pa="BA"=D[5], 2 e Pb="ABBA"=D[6]

Passo 6b: Enviar para a saída código de Pa: (5,2)

Passo 6c: Padrão candidato C1="BAABBA" -> Novo padrão, então D[7]="BAABBA"

Passo 6d: Pa=Pb



EXEMPLO: Codificação LZW-HX

Sequência a codificar (K=2): "AAABABBBABABAABBABBBAB"

Passo 3a: Pa="AB"=D[4], 2 e Pb="BA"=D[5]

Passo 3b: Enviar para a saída código de Pa: (4,2)

Passo 3c: Padrão candidato C1="ABBA" -> Novo padrão, então D[6]="ABBA"

Passo 3d: Pa=Pb

Passo 4a: Pa="BA"=D[5] e Pb="BA"=D[5]

Passo 4b: Enviar para a saída código de Pa: (5)

Passo 4c: Padrão candidato C1="BABA" -> Pode ocupar lugar de "BA", então D[5]="BABA"

Passo 4d: Pa=Pb

Passo 5a: Pa="BA"=D[5], 2 e Pb="BA"=D[5], 2

Passo 5b: Enviar para a saída código de Pa: (5,2)

Passo 5c: Padrão candidato C1="BABA" -> Já está definido em D[5]

Passo 5d: Pa=Pb

Passo 6a: Pa="BA"=D[5], 2 e Pb="ABBA"=D[6]

Passo 6b: Enviar para a saída código de Pa: (5,2)

Passo 6c: Padrão candidato C1="BAABBA" -> Novo padrão, então D[7]="BAABBA"

Passo 6d: Pa=Pb



EXEMPLO: Codificação LZW-HX

Sequência a codificar (K=2): "AAABABBABABAABBABBAB"

Passo 7a: Pa="ABBA"=D[6] e Pb="BB"=D[3]

Passo 7b: Enviar para a saída código de Pa: (6)

Passo 7c: Padrão candidato C1="ABBABB" -> Pode ocupar lugar de "ABBA", então D[6]="ABBABB"

Passo 7d: Pa=Pb

Passo 8a: Pa="BB"=D[3] e Pb="AB"=D[4],2

Passo 8b: Enviar para a saída código de Pa: (3)

Passo 8c: Padrão candidato C1="BBAB" -> Pode ocupar lugar de "BB", então D[3]="BBAB"

Passo 8d: Pa=Pb

Passo 9a: Pa="AB"=D[4],2 e Pb=""

Passo 9b: Enviar para a saída código de Pa: (4,2)

Passo 9c: Fim*

Output: (2) (4) (4,2) (5) (5,2) (5,2) (6) (3) (4,2)



EXEMPLO: Codificação LZW-HX

	P_a	P_b	Candidato C_i	Ação no Dicionário	Saída
1	"AA"=D[2]	"AA"=D[4]	C_1 ="AAAB" pode ocupar lugar de "AA"	$D[2]$ ="AAAB"	(2)
2	"AB"=D[4]	"AB"=D[4]	C_2 ="ABAB" pode ocupar o lugar de "AB"	$D[4]$ ="ABAB"	(4)
3	"AB"=D[4]	"BA"=D[5]	C_3 ="ABBA" novo padrão	$D[6]$ ="ABBA"	(4,2)
4	"BA"=D[5]	"BA"=D[5]	C_4 ="BABA" pode ocupar o lugar de "BA"	$D[5]$ ="BABA"	(5)
5	"BA"=D[5],2	"BA"=D[5],2	C_5 ="BABA" já existe em $D[5]$	-	(5,2)
6	"BA"=D[5],2	"ABBA"=D[6]	C_6 ="BAABBA" novo padrão	$D[7]$ ="BAABBA"	(5,2)
7	"ABBA"=D[6]	"BB"=D[3]	C_7 ="ABBABB" pode ocupar o lugar de "ABBA"	$D[6]$ ="ABBABB"	(6)
8	"BB"=D[3]	"AB"=D[4],2	C_8 ="BBAB" pode ocupar o lugar de "BB"	$D[3]$ ="BBAB"	(3)
9	"AB"=D[4],2	""	Não há	-	(4,2)